

passgen

Детерминированный генератор паролей

Документация для защиты проекта

Подробное описание всех возможностей программы

1. Что такое passgen и зачем он нужен?

Представьте себе ситуацию: у вас есть аккаунты на десяти разных сайтах — вконтакте, гугл, гитхаб, телеграм, почта, онлайн-банкинг, форумы, учебные порталы и так далее. Для каждого сайта нужен пароль. Использовать один и тот же пароль везде — это очень опасно: если злоумышленник узнает пароль от одного сайта, он сможет зайти на все остальные. Запоминать тридцать разных сложных паролей невозможно — человеческая память на такое не рассчитана.

Обычно люди решают эту проблему с помощью менеджеров паролей — специальных программ, которые хранят все пароли в зашифрованном виде. Но у этого подхода есть недостаток: все пароли хранятся в одном месте. Если злоумышленник взломает менеджер паролей, он получит доступ ко всем аккаунтам сразу. Кроме того, менеджеры паролей — это отдельные программы, которые нужно устанавливать, настраивать, и часто они платные.

Программа passgen предлагает принципиально другой подход. Вместо того чтобы хранить пароли, она вычисляет их каждый раз заново с помощью криптографического алгоритма. Вам нужно запомнить всего одну вещь — секретную фразу (сид). Например: "Моя самая секретная фраза 2024". Зная эту фразу и название сайта (например, "google"), программа сможет восстановить пароль в любой момент.

Это работает как математическая формула: если вы знаете сид и название сервиса, то пароль всегда будет одинаковым. Не нужно ничего хранить, нечего красть, невозможно потерять. Забыли пароль от гугла? Просто введите сид и слово "google" — программа выдаст тот же самый пароль, который был в первый раз.

2. Как именно программа создаёт пароль?

Это самый важный раздел для понимания программы. Здесь объясняется криптографический алгоритм шаг за шагом, простыми словами.

Шаг 1. Пользователь вводит сид

Сид — это любая фраза, которую пользователь придумывает и запоминает. Это может быть строка из букв и цифр, предложение на русском или английском, набор слов — что угодно. Программа берёт эту фразу и превращает её в последовательность байтов (чисел от 0 до 255). Эта последовательность становится секретным ключом для дальнейших вычислений. Без этого ключа восстановить пароль невозможно, поэтому сид нужно держать в секрете и не забывать.

Шаг 2. Формируется сообщение для алгоритма

Программа собирает вместе три вещи: название сервиса (например, "google"), желаемую длину пароля (например, 16 символов) и номер шага. Номер шага начинается с нуля и увеличивается каждый раз, когда программе нужно больше случайных данных. Всё это объединяется в одну строку и превращается в байты. Получается сообщение, которое будет обработано криптографическим алгоритмом.

Шаг 3. Вычисление HMAC-SHA256

HMAC-SHA256 — это международный криптографический стандарт, который используется в самых разных областях. Например, в банковских системах для подтверждения транзакций, в блокчейне биткоина для создания цифровых подписей, в протоколе Wi-Fi для защиты беспроводной сети. Алгоритм берёт два входных параметра — ключ (сид пользователя) и сообщение (сервис+длина+шаг) — и вычисляет 32 байта (256 бит) данных.

У этого алгоритма есть два важнейших свойства. Первое: результат невозможно предсказать, не зная ключа. Даже если изменить всего один символ в сиде или в названии сервиса, результат изменится полностью. Второе: для одинаковых входных данных результат всегда одинаковый. Это свойство называется детерминированностью. Благодаря ему можно восстановить пароль в любой момент — достаточно ввести те же самые данные.

Шаг 4. Превращение байтов в символы пароля

После вычисления HMAC программа получает 32 байта случайных чисел. Теперь нужно превратить эти числа в символы, из которых будет состоять пароль. Для этого программа смотрит, какие наборы символов выбрал пользователь (заглавные буквы, строчные, цифры, спецсимволы), и составляет из них один общий алфавит. Затем каждый байт проверяется: если его значение подходит (меньше определенного порога), он превращается в символ из алфавита. Например, если алфавит состоит из 50 символов, а байт равен 123, программа вычисляет 123 разделить на 50, берёт остаток от деления (23) и выбирает 23-й символ из алфавита.

Почему используется проверка на порог? Это называется rejection sampling. Дело в том, что если просто брать остаток от деления для всех байтов, то некоторые символы будут появляться чаще других (из-за того, что 256 не делится нацело на количество символов в алфавите). Чтобы этого избежать, программа отбрасывает байты, которые выходят за пределы максимального значения. Это делает распределение символов равномерным, и пароль становится криптографически стойким.

Если в одном блоке из 32 байтов не хватило подходящих значений для создания пароля нужной длины, программа увеличивает номер шага и вычисляет следующий блок HMAC. Процесс повторяется до тех пор, пока не наберётся нужное количество символов.

Шаг 5. Дополнительные преобразования

Если пользователь включил опцию "Первая заглавная", программа берёт первый символ полученного пароля и делает его заглавным (если он строчный). Если включена опция "Последняя цифра", программа вычисляет дополнительный HMAC специально для последней цифры (используя другие входные данные, чтобы не нарушить детерминированность) и заменяет последний символ пароля на полученную цифру. Эти опции нужны потому, что многие сайты и сервисы требуют, чтобы пароль содержал хотя бы одну заглавную букву и хотя бы одну цифру.

3. Почему программа безопасна?

На защите проекта могут спросить про безопасность. Вот основные аргументы:

Аргумент 1. Данные не покидают компьютер пользователя

Вся программа работает на локальном компьютере. Сервер запускается на адресе 127.0.0.1 (localhost) — это специальный адрес, который означает "этот же компьютер". Никакие данные не отправляются в интернет. Ни сид, ни пароль, ни названия сервисов не передаются по сети. Даже веб-интерфейс открывается с локального сервера, а не из интернета. Это значит, что злоумышленник не может перехватить данные при передаче.

Аргумент 2. Используется проверенный криптографический стандарт

Программа не изобретает свой собственный алгоритм шифрования — это всегда опасно, потому что самодельные алгоритмы часто содержат ошибки и уязвимости. Вместо этого passgen использует HMAC-SHA256 — международный стандарт, который прошёл тысячи проверок специалистами по криптографии со всего мира. Этот же алгоритм используется в банковских системах, в blockchain, в защите Wi-Fi сетей. Если он безопасен для банков, то безопасен и для паролей.

Аргумент 3. Пароли не хранятся — их нечего красть

В отличие от менеджеров паролей, которые хранят базу данных со всеми паролями, passgen не хранит пароли вообще. Пароль каждый раз вычисляется заново по запросу. Если злоумышленник получит доступ к компьютеру, он найдёт только файлы самой программы (app.py и index.html), но не найдёт ни одного пароля. Даже настройки (длина, типы символов) не сохраняются между запусками программы — каждый запуск начинается с чистого листа.

Аргумент 4. Равномерное распределение символов (rejection sampling)

Программа использует специальную технику для обеспечения равномерного распределения символов в пароле. Если бы этой техники не было, некоторые символы появлялись бы в паролях чаще, чем другие. Злоумышленник мог бы этим воспользоваться и угадывать пароль быстрее, перебирая сначала самые вероятные символы. Rejection sampling устраняет эту возможность.

Аргумент 5. Для каждого сервиса — уникальный пароль

Название сервиса является частью сообщения, которое подаётся на вход алгоритму HMAC. Поэтому пароль для "google" полностью отличается от пароля для "goggle" (разница всего в одну букву) и тем более от пароля для "github". Это означает, что даже если злоумышленник каким-то образом узнал пароль от одного сервиса, он не сможет использовать его для входа в другие сервисы.

4. Из каких частей состоит программа?

Программа состоит из двух файлов: `app.py` и `index.html`. Они работают вместе, но выполняют разные задачи.

Файл `app.py` — серверная часть (Python)

Файл `app.py` написан на языке Python. Его задача — запустить небольшой веб-сервер на компьютере пользователя. Этот сервер делает только две вещи: отдаёт файл `index.html` браузеру по запросу, и открывает окно браузера в специальном режиме. Сервер автоматически находит свободный порт (начиная с 8765), чтобы не было конфликтов с другими программами. В лог сервер ничего не пишет, чтобы не засорять терминал.

Режим открытия окна: программа пытается открыть браузер с параметром `--app`, который создаёт отдельное окно без адресной строки, закладок и панелей — выглядит как настоящее приложение. Если такой браузер не найден, открывается обычное окно браузера. Программа проверяет несколько браузеров: Google Chrome, Chromium, Brave, Vivaldi, Microsoft Edge, Firefox. Сервер работает в фоновом потоке, а главная программа ждёт нажатия `Ctrl+C` для завершения.

Файл `index.html` — клиентская часть (HTML+CSS+JS)

Это главный файл программы. В нём содержится всё, что видит пользователь: поля ввода, кнопки, списки — и вся логика работы: генерация паролей, сохранение настроек, экспорт и импорт. Файл состоит из трёх частей, которые тесно связаны между собой.

HTML (разметка): описывает структуру страницы — блоки (карточки) с полями, кнопками и надписями. Каждый элемент интерфейса находится внутри своего блока. Например, блок с сидом, блок с сервисом, блок с настройками, блок с результатом.

CSS (стили): отвечает за внешний вид программы. Цвета (фиолетово-белая гамма), шрифты, закругления углов, тени, анимации. Все цвета заданы через специальные переменные в начале файла, чтобы их было легко менять. Анимации делают программу приятнее: элементы плавно появляются, кнопки слегка уменьшаются при нажатии, панели раскрываются и сворачиваются.

JavaScript (логика): содержит все функции программы. Использует встроенный в браузер Web Crypto API — это библиотека криптографических функций, которая есть в любом современном браузере (Chrome, Firefox, Edge, Safari). Она позволяет вычислять HMAC-SHA256 прямо на устройстве пользователя, без отправки данных в интернет. Весь код JavaScript написан в одном файле внутри тега `<script>`.

5. Подробное описание каждой функции программы

В этом разделе объясняется каждая функция программы. Описания написаны так, чтобы человек, который вообще не видел код, смог прочитать и понять, что делает программа, как она работает и для чего нужна каждая её часть.

5.1. Функция `init()` — подготовка программы к работе

Когда пользователь открывает страницу (или программа запускается), должна сработать `init()`. Это как нажать кнопку включения — все системы должны подготовиться к работе. Эта функция делает несколько важных дел.

Во-первых, она очищает все старые данные. Если пользователь уже запускал программу раньше, в браузере могли остаться какие-то сохранённые настройки. `init()` удаляет их — это сделано для безопасности, чтобы каждый запуск начинался с чистого листа.

Во-вторых, она привязывает к полям ввода специальные обработчики. Например, если пользователь нажал `Enter` в поле сیدا или сервиса — должна запуститься генерация пароля. Если пользователь начал вводить сід после того, как была показана ошибка — ошибка должна исчезнуть. Если пользователь переключился на другое поле после ввода сервиса — программа должна запомнить, какой сервис сейчас выбран.

В-третьих, она проверяет, есть ли сохранённые настройки для сервиса, который был выбран в прошлый раз (в течение текущей сессии). Если есть — восстанавливает их. Если нет — устанавливает стандартные настройки: заглавные и строчные буквы, длина 10 символов.

В-четвёртых, она отрисовывает (создаёт на экране) список сервисов с логотипами, чтобы пользователь мог видеть, какие сервисы доступны.

5.2. Функция `loadState()` — очистка всех данных

Эта функция вызывается в самом начале, при запуске программы. Её задача — полностью очистить все данные, которые могли остаться от предыдущего использования. Она удаляет всё, что могло сохраниться в памяти браузера (`localStorage`), обнуляет список сервисов, сбрасывает настройки на стандартные, устанавливает длину пароля 10 и очищает поле ввода сервиса.

Эта функция — часть системы безопасности. Благодаря ей, если вы закрыли программу и открыли снова, никаких следов предыдущего использования не останется. Даже названия сервисов, для которых вы генерировали пароли, не сохранятся между сессиями. Это защищает вашу конфиденциальность.

5.3. Функция `saveCfg()` — сохранение настроек для сервиса

Представьте: вы настроили пароль для гугла — длину 16 символов, все типы символов, первая заглавная, последняя цифра. Через час вы снова выбираете гугл из списка — и программа сама восстанавливает эти настройки. Это работает благодаря `saveCfg()`.

Когда пользователь нажимает кнопку "Создать" (сгенерировать пароль), `saveCfg()` запоминает, какие настройки сейчас активны: какие чекбоксы включены (заглавные, строчные, цифры, спецсимволы), какая длина пароля, какие опции инверсии включены. Всё это сохраняется в объекте `perSvc` — специальной области памяти, где хранятся настройки для каждого сервиса. Настройки сохраняются под названием текущего сервиса (например, "google").

Важная особенность: настройки сохраняются только в оперативной памяти, а не на диске. При закрытии программы они исчезают. Это сделано для безопасности и чистоты сессии.

5.4. Функция `applyCfg(cfg)` — восстановление настроек сервиса

Когда пользователь выбирает сервис из списка (например, кликает на "github"), программа должна вспомнить, какие настройки использовались для этого сервиса, и применить их. Это делает `applyCfg()`.

Функция принимает один параметр — `cfg`, объект с сохранёнными настройками. Она устанавливает длину пароля, восстанавливает состояние всех четырёх чекбоксов, включает или выключает опции инверсии (первая заглавная, последняя цифра). После этого пользователь видит те же настройки, которые были при последней генерации пароля для этого сервиса.

5.5. Функция `toggleInv(id)` — переключение опций инверсии

Опции инверсии — это "Первая заглавная" и "Последняя цифра". Они находятся в расширенных настройках, потому что используются не так часто. Когда пользователь кликает на одну из этих опций, `toggleInv()` переключает её состояние: если было выключено — включает, если было включено — выключает.

После переключения функция обновляет внешний вид кружочка на экране: если опция активна — кружок закрашивается фиолетовым, если нет — остаётся пустым. Сами настройки при этом не сохраняются — сохранение произойдёт только при нажатии кнопки "Создать".

5.6. Функция `applyPreset()` — выбор готового пресета

В программе есть выпадающий список с тремя готовыми вариантами настроек: "Минимальный" (8 символов, заглавные+строчные+цифры), "Стандартный" (10 символов, заглавные+строчные+цифры), "Максимальный" (16 символов, все символы). Когда пользователь выбирает один из них, `applyPreset()` применяет соответствующие настройки.

Функция проверяет, какой пункт выбран, и в зависимости от этого устанавливает длину пароля и включает нужные наборы символов. После применения настроек функция обновляет чекбоксы на экране и сохраняет настройки для текущего сервиса. Пункт "Свой" означает, что пользователь хочет настроить всё вручную — при его выборе ничего не происходит.

5.7. Функция `renderChk()` — отрисовка чекбоксов символов

На экране есть четыре кружочка (чекбокса) для выбора наборов символов: заглавные буквы, строчные буквы, цифры, спецсимволы. Функция `renderChk()` создаёт эти кружочки и размещает их на экране.

Для каждого набора символов функция создаёт строку с кружочком и названием. Если набор активен — кружочек закрашивается фиолетовым, если нет — остаётся пустым (только обводка). При клике на строку значение переключается: если было включено — выключается, и наоборот. После переключения функция запускается снова, чтобы обновить изображение.

5.8. Функция `adj(delta)` — изменение длины пароля

Рядом с полем длины пароля есть две маленькие стрелочки: вверх и вниз. Функция `adj()` обрабатывает клики по этим стрелочкам. Она принимает параметр `delta`: +1 для увеличения длины на 1 и -1 для уменьшения.

Функция берёт текущее значение длины из поля, прибавляет к нему `delta`, и записывает результат обратно в поле. При этом она проверяет, чтобы длина не стала меньше 1 и не превысила 99 — это минимальное и максимальное допустимые значения.

5.9. Функция `toggleSvcPanel()` — открытие и закрытие списка сервисов

Справа от поля ввода сервиса есть кнопка "список" со стрелочкой. При клике на неё открывается или закрывается панель со списком сервисов. `toggleSvcPanel()` управляет этим.

Функция просто переключает CSS-класс на панели: если он есть — убирает (панель закрывается), если нет — добавляет (панель открывается с плавной анимацией раскрытия). Стрелочка на кнопке при этом поворачивается, показывая, открыт список или закрыт.

5.10. Функция `renderSvcChips()` — сборка списка сервисов

Эта функция собирает вместе все сервисы и отображает их в панели списка. Сначала она показывает популярные сервисы (Google, GitHub, VK, Яндекс, Mail, iCloud, Dropbox, Telegram) — но только те из них, которые пользователь ещё не добавлял в свои сохранённые. Затем показывает все сервисы, которые пользователь уже использовал в текущей сессии.

Такое разделение нужно, чтобы популярные сервисы всегда были под рукой, но не дублировались, если пользователь уже добавил их в свои сохранённые.

5.11. Функция `chip()` — создание одной кнопки сервиса

Каждый сервис в списке отображается в виде чипа — небольшой кнопки с названием и логотипом. Функция `chip()` создаёт один такой чип.

Если сервис из популярного списка, функция создаёт цветной кружок с буквой — например, синий кружок с буквой G для Google, чёрный с буквами GH для GitHub, синий с буквой V для VK. Если сервис добавлен пользователем (его нет в популярном списке), создаётся серый кружок с вопросительным знаком.

При клике на чип программа заполняет поле сервиса его названием и, если есть сохранённые настройки для этого сервиса, применяет их. Если у сервиса есть правая кнопка мыши (контекстное меню), то при клике правой кнопкой сервис удаляется из списка.

5.12. Функция `gen()` — генерация пароля (самая главная функция)

Это самая важная функция во всей программе. Ради неё всё и задумывалось. Она запускается, когда пользователь нажимает кнопку "Создать" или нажимает Enter в полях сида или сервиса.

Сначала функция проверяет, ввёл ли пользователь сид. Если поле сида пустое — функция показывает ошибку: поле подсвечивается розовым цветом, и появляется сообщение "Введите сид". Генерация прерывается. Это сделано потому, что без сида пароль будет бессмысленным — его невозможно будет восстановить.

Затем функция проверяет, выбран ли хотя бы один набор символов. Если пользователь отключил все четыре чекбокса — тоже показывает ошибку и прерывает генерацию.

После всех проверок функция берёт выбранные наборы символов и объединяет их в один общий алфавит. Например, если выбраны заглавные и цифры, алфавит будет выглядеть как "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789".

Затем функция вызывает `hmacGen()`, передавая ей сид, название сервиса, длину пароля и алфавит. `hmacGen()` вычисляет пароль с помощью HMAC-SHA256.

После получения пароля функция применяет опции инверсии: если включена "Первая заглавная" — делает первый символ заглавным. Если включена "Последняя цифра" — вычисляет цифру через отдельный HMAC и заменяет последний символ.

Готовый пароль отображается в специальном поле и автоматически копируется в буфер обмена (чтобы пользователь мог сразу вставить его на сайте). Функция также обновляет индикатор надёжности, добавляет сервис в список (если его там ещё нет) и сохраняет настройки.

5.13. Функция `hmacGen()` — криптографическое ядро программы

Эта функция — самое сердце программы. Именно здесь происходит вся криптография. Она принимает четыре параметра: сид (секретную фразу), название сервиса, желаемую длину пароля и алфавит (список символов, из которых можно составлять пароль).

Функция работает в цикле: пока не наберётся нужная длина пароля, она вычисляет HMAC-SHA256 от сообщения, которое состоит из названия сервиса, длины пароля и номера шага. Номер шага (счётчик) начинается с 0 и увеличивается на 1 после каждого шага. Это гарантирует, что каждый следующий блок будет уникальным.

Для вычисления HMAC используется Web Crypto API — встроенная в браузер криптографическая библиотека. Она доступна в любом современном браузере и не требует установки дополнительных программ.

Каждый вызов HMAC даёт 32 байта (256 бит) данных. Для каждого байта функция проверяет, подходит ли он: если байт меньше максимального значения (256 минус остаток от деления 256 на длину алфавита), то он используется. Затем байт делится на длину алфавита, и остаток от деления указывает на символ в алфавите. Если байт не подходит — он пропускается. Когда функция набирает достаточно символов, она возвращает готовый пароль.

5.14. Функция `genLogin()` — генерация логина

Многие сайты просят придумать не только пароль, но и имя пользователя (логин). `genLogin()` создаёт короткий логин из 6 символов, который легко запомнить.

Для генерации используется SHA-256 — ещё один криптографический алгоритм, родственник HMAC. Функция берёт сид, добавляет к нему слово "login" и название сервиса, прибавляет значение счётчика (который увеличивается при каждом вызове) и вычисляет хеш SHA-256. Из полученных 32 байтов берутся первые 6 и превращаются в символы.

Алфавит для логина специальный: в нём только строчные буквы, но исключены i, l, o (их можно перепутать с цифрами 1 и 0), а также цифры от 2 до 9 (исключены 0 и 1, потому что они похожи на буквы). В результате получается логин вроде "abc34x", который легко продиктовать по телефону и невозможно перепутать.

Благодаря счётчику, который увеличивается при каждом нажатии, каждый следующий клик по кнопке "Создать логин" даёт новый, отличающийся логин.

5.15. Функция `updateStrength()` — оценка надёжности пароля

После генерации пароля функция `updateStrength()` оценивает, насколько он надёжен, и показывает результат в виде трёх полосок и текстовой рекомендации.

Оценка основана на двух параметрах: длина пароля и количество выбранных наборов символов. Если пароль короче 10 символов или выбрано меньше 3 наборов — он считается слабым (одна розовая полоска, рекомендуется увеличить длину или добавить типы символов). Если длина от 10 символов и выбрано 3+ набора — средний (две малиновые полоски). Если длина от 14 символов и выбраны все 4 набора — отличный (три сиреневые полоски).

Эта функция даёт пользователю мгновенную обратную связь: он сразу видит, нужно ли улучшить пароль, или он уже достаточно надёжен.

5.16. Функция `cpy()` — копирование пароля в буфер обмена

Когда пользователь нажимает кнопку "Копировать" (или после автоматической генерации), функция `cpy()` копирует пароль в буфер обмена.

Она использует современный метод `navigator.clipboard.writeText()`, который работает во всех современных

браузерах. Если этот метод недоступен (старый браузер), функция использует запасной способ — выделяет текст в поле и выполняет команду "Копировать". После копирования на экране появляется сообщение "Скопировано!" на 1.5 секунды.

5.17. Функция `exportData()` — экспорт настроек в файл

Если пользователь хочет перенести свои настройки на другой компьютер или просто сохранить их, он может нажать "Экспорт". Функция `exportData()` собирает все сервисы, для которых есть сохранённые настройки, и создаёт JSON-файл.

JSON — это текстовый формат, который легко читается и человеком, и компьютером. Файл содержит список сервисов и для каждого — настройки: какие символы включены, опции инверсии, длина пароля. Файл автоматически скачивается в папку загрузок.

5.18. Функция `importData()` — импорт настроек из файла

Чтобы загрузить ранее сохранённые настройки, пользователь нажимает "Импорт" и выбирает JSON-файл. Функция `importData()` читает файл, проверяет, что это действительно файл `passgen` (правильный формат), и если всё в порядке — открывает окно выбора сервисов.

Окно выбора нужно, чтобы пользователь мог импортировать только те сервисы, которые ему нужны, а не все сразу. Каждый сервис отображается с галочкой (по умолчанию включена). Пользователь снимает галочки с ненужных сервисов и нажимает "Импортировать".

5.19. Функция `showImportModal()` — окно выбора сервисов

Эта функция создаёт и показывает модальное окно (окно поверх программы) со списком сервисов из импортируемого файла. Для каждого сервиса создаётся строка с названием и чекбоксом. Все чекбоксы по умолчанию включены. Если в файле нет ни одного сервиса — показывается сообщение об ошибке.

5.20. Функция `confirmImport()` — подтверждение импорта

Когда пользователь отметил нужные сервисы и нажал "Импортировать", `confirmImport()` загружает выбранные настройки в программу. Для каждого отмеченного сервиса она восстанавливает настройки (длину, символы, инверсию) и добавляет сервис в список, если его там ещё не было.

После завершения импорта окно закрывается, список сервисов обновляется, и показывается сообщение "Импорт завершён!" на 3 секунды.

5.21. Функции `closeImportModal()` и `toggleAdvanced()`

`closeImportModal()` — закрывает окно импорта, если пользователь передумал. Очищает временные данные и просто убирает окно с экрана.

`toggleAdvanced()` — открывает или закрывает панель расширенных настроек (опции "Первая заглавная" и "Последняя цифра"). Панель скрыта по умолчанию, чтобы не занимать место, и показывается только по клику на "Расширенные". Открывается и закрывается с плавной анимацией.

6. Элементы интерфейса: что для чего

Поле "Сид" — самое важное поле. Сюда вводится секретная фраза. Без неё невозможно восстановить пароль. Если поле пустое — программа подсвечивает его розовым и показывает ошибку. Нажатие Enter в этом поле запускает генерацию пароля.

Поле "Сервис" — сюда вводится название сайта (google, github, vk). От названия зависит, какой пароль получится. Справа кнопка "список" с популярными сервисами. У каждого популярного сервиса есть цветной кружок-логотип. Если своего сервиса нет в списке — его можно ввести вручную.

Длина пароля — сколько символов будет в пароле. Рекомендуется от 10 до 16. Меньше 8 — небезопасно, больше 32 — неудобно вводить. Стрелочки для быстрой подстройки.

Четыре кружочка — выбор типов символов: заглавные буквы, строчные, цифры, спецсимволы. Включено — кружок фиолетовый, выключено — пустой. Нужно выбрать хотя бы один.

Пресеты (Минимальный/Стандартный/Максимальный) — готовые варианты настроек одним кликом.

Расширенные настройки — "Первая заглавная" (делает первую букву большой) и "Последняя цифра" (заменяет последний символ на цифру). Спрятаны, чтобы не мешать.

Поле логина — генерирует короткий логин из 6 символов. Каждый клик — новый логин.

Полоска надёжности — три полоски под паролем. Розовый (слабый) -> малиновый (средний) -> сиреневый (отличный). Показывает, насколько хорош пароль.

Кнопки: "Создать" (главная — запускает алгоритм), "Копировать" (копирует пароль), "Создать логин" (рядом с полем логина), "Экспорт" (скачивает настройки), "Импорт" (загружает настройки).

7. Установка и запуск

Программа работает на любой операционной системе: Windows, Linux, macOS. Главное требование — установленный Python 3 (любая версия от 3.6 и выше).

Linux (Arch, Ubuntu и другие):

1. Скачиваем программу:

```
git clone ...  
cd xxxulay
```

2. Запускаем:

```
python3 app.py
```

Windows:

1. Скачиваем ZIP-архив с GitHub (кнопка Code -> Download ZIP)

2. Распаковываем архив в любую папку

3. Открываем эту папку и запускаем:

```
python app.py
```

Что происходит после запуска?

1. В терминале появляется сообщение "passgen — http://localhost:..."
2. Автоматически открывается отдельное окно с программой (без адресной строки)
3. Если окно не открылось — переходим по адресу из терминала в любом браузере
4. Чтобы выйти — нажимаем Ctrl+C в терминале

Дополнительные программы не требуются

Не нужно ничего устанавливать. Сервер работает на встроенных библиотеках Python (http.server, socketserver). Криптография — на встроенном Web Crypto API браузера. Интерфейс — на HTML/CSS/JavaScript, которые есть в любом браузере.